might lead to pull requests (PRs) that need approval and failures. If that becomes the case, quality checking should be made part of the data products themselves. Only the results of the checks would then be exposed to a GCS location and picked up by the central component to get pushed into DataHub.

Federated computational governance is discussed only briefly in this architecture, but with the central DataHub, and specifically a GUI for registering data products, as well as the central component running checks on the datasets itself, there is a lot of space to inject automatically executable policies into this platform.

Data as a product in this architecture hinges a lot on each team having easy ways of manipulating its data. We believe Dataflow serves that purpose in this example.

Data domain ownership in this architecture ironically hinges on the service accounts and the access control, because the architecture is using a large shared service. It is critically important to have technological boundaries to access; otherwise, the ownership would not truly fall into the hands of the data-producing teams.

8.1.7 GCP architecture summary

This platform architecture uses shared cloud services, mostly native to the GCP environment. It uses a few components that are not GCP specific, Git and Terraform, and in the variations, custom code and DataHub.

This architecture provides data producers with a set of tooling for transforming data, but only one standardized data port. The data ports, however, could be easily extended to use Pub/Sub and GCS as well, giving you a large range of target formats and satisfying most consumer needs.

The setup can be translated, with minor differences, to the other large cloud providers AWS and Microsoft Azure as well. Since they would look similar, we chose not to display them here as a one-to-one mapping. Table 8.1 summarizes the GCP architecture profile.